云原生技术综合实验：从虚拟化到容器编排的消融对比

一、实验概述

本次实验用纯 Python 标准库实现一套 Paxos 协议原型，在同一套代码里提供 Basic Paxos / Multi-Paxos / Fast Paxos 三种提案机制，支持 3~5 个节点、TCP 自定义报文、Acceptor 持久化、崩溃恢复、超时重试、批处理与流水线，并内置可注入 RTT/丢包/网络分区的传输层。

在此基础上完成了 52 个用例的对比实验与 8 个故障/分区场景，采集了 P50/P95/P99 延迟、吞吐、消息数、冲突率、恢复时间等指标，并验证了安全性。

本次实验工程已开源至 https://github.com/koshistar/paxos-lab。

二、环境与参数

1. 软硬件环境

项目	配置
宿主机型号	L16
CPU	AMD Ryzen 9 7945HX（16 核 32 线程，2.5GHz 基准）
内存	16.0GB DDR5
操作系统	Windows 11
GPU	NVIDIA GeForce RTX 5070 Ti Laptop
Python	3.13.9

2. 关键参数

参数	默认值	含义
--protocol	multipaxos	basic / multipaxos / fast
--fsync	on	Acceptor 每条记录 flush+os.fsync；off 时退化为用户态缓冲
--rtt-ms	1	往返时延，代码注入单程 rtt/2
--loss	0	发送方向丢包概率
--batch-ms / --batch-size	0/64	批处理窗口与单批上限
--max-inflight	16	同时在途的批次数
--heartbeat-ms / --elsection-ms	60/400	Leader 心跳间隔 / 选举超时
尝试超时	max(300ms,10*RTT+100ms)	单轮 RPC 超时
客户端超时	max(2s,12*RTT+0.5s)	客户端感知超时

3. 多数派与仲裁大小

节点数 n	经典多数派 Q=⌊ n/2⌋ +1	可容忍故障 n-Q	Fast Paxos 快速仲裁 F=⌈ 3n/4⌉	说明
3	2	1	3	F 等于全体，任何体态慢或挂都会毁掉快速路径
4	3	1	3	F 真正开始有意义
5	3	2	4	容错提升到 2

				台
6	4	2	5	浪费一台机器
7	4	3	6	常见上限

4. 实测 RTT 与设定 RTT 的偏差

设定 RTT	实测 RTT	偏差
1ms	1.64~1.81ms	+0.6~0.8ms
10ms	10.78~10.97ms	+0.8~1.0ms
50ms	50.76~52.58ms	+0.8~2.6ms
100ms	100.89~108.36ms	+0.9~8.4ms

三、开源项目架构分析

本次实验选择 etcd。

1. 整体架构分层

etcd 把“共识 → 日志 → 状态机 → 存储引擎”彻底解耦，外面包一层 gRPC。

客户端 client/v3：连接管理、重试、事务封装。

API 层 server/etcd/api/、v3_server.go：协议翻译、鉴权、限额、挂载流式 Watch。

etcdserver server/etcdserver/server.go：串起提案、apply、配额警告、选主后的租约晋升。

raft raft/：纯算法，不碰网络与磁盘，输入消息与 tick，输出 Ready。

WAL server/storage/wal/：raft 的硬盘记忆，崩溃靠它重放。

MVCC server/storage/mvcc/：真正的状态机，key 带 revision，可读历史、可 compact。

bbolt server/storage/backend/：B+树引擎，提供页级事务与批量提交。

2. 关键代码导读

路径	负责什么
server/embed/etcd.go	进程装配。StartEtcd 校验配置、建 peer/client/metrics 监听、把 Config 翻成 etcdserver 的 ServerConfig。源码中可见 UnsafeNoFsync、PreVote 在此传递
Servere/etcdserver/server.go	服务主体。持有 raftNode、MVCC、lessor、配额告警，把请求变成提案、把已提交条目驱动到 apply，选主后把本地 lessor 提升为 primary 接管租约回收。
server/etcdserver/v3_server.go	对外 RPC 实现。写请求走“提案 → 等结果”，需要线性一致的 Range 走 ReadIndex。
Server/etcdserver/raft.go	粘合层。RaftNode 跑 ticker+Ready()循环，更新 leader、派发 ReadStates、把已提交条目交给 apply，再 SaveSnap/Save/Advance。
raft/	共识本体。Step(msg)推状态、Tick()推时钟、Ready()吐出待持久化与待发送内容，含选举、MsgPreVote、批量、流水线、ReadIndex、失去多数派自动退位，MustSync 判断是否必须 sync。
server/storage/wal/	预写日志。文件名$seq-$index.wal，超 64MB 切分，每条记录带 64 位长度头+CRC32，Save 追求状态与条目，SaveSnapshot 记快照边界，ReadAll 启动读回。
server/storage/mvcc/	MVCC。key 按 revision 存多版本，内存 treeIndex 管版本链，key-value 在 backend，store 维护 currentRev/compactMainRev，restore 重建索引，compaction 回收旧版本。

server/storage/backend/、schema/	bbolt 封装与元数据布局。读、写、批量事务，以及 lease、key、meta 的 bucket 常量。
Server/lease/	租约。leassor 管 Grant/Revoke/Renew/Attach，primary lessor 就是 raft leader，只持久化 lease ID 与 TTL，挂载的 key 从 KV 反查，换主时把剩余 TTL checkpoint 进共识日志。

3. 持久化与崩溃恢复

WAL 的作用：raft 要求"投过票、承诺过的日志"不能因重启消失，WAL 就是这条承诺的载体——先写日志，再改内存，最后才说"已提交"。etcd 的 WAL 分段（$seq-$index.wal），超 64MB 切分，每条记录带 CRC，末段损坏最多丢末尾不完整记录。

snapshot 的作用：只留 WAL 会无限增长，重启要重放全部历史。于是定期把完整状态做成快照并在 WAL 记边界，--snapshot-count（默认 100000 个已提交事务）是阈值；有了快照，旧 WAL 段可释放，新成员可直接收快照追赶。恢复流程即 snapshot → 重建 MVCC 索引 → 重放其后的 WAL。顺序有硬约束，raftNode 注释写得很直白：必须先保存 snapshot 文件与 WAL 里的快照记录，再保存任何 entries 与 hardstate。

--unsafe-no-fsync 意味着什么：关掉 WAL 的 fsync，写入只到页缓存，"committed 即持久"不再成立——掉电时已回复客户端的写可能消失，甚至回退到更旧的 term/commit。官方文档把这类参数单列为 Unsafe flags，明确警告"会破坏共识协议给出的保证"。它只为对照实验存在。

4. 会话与租约

Chubby 的核心抽象是 session + 锁：session 活着锁就活着，session 过期锁自动释放。陷阱是客户端可能"以为自己还持锁"，所以 Chubby 用 sequencer 让下游能校验凭证是否过期。

etcd 没有这两个原语，用三件东西等价拼出：

lease（TTL）= session 生命周期：LeaseGrant 给 TTL，客户端须周期 LeaseKeepAlive，否则到期回收。

事务（Txn+Compare）=加锁的原子性：比较 create_revision==0，只有 key 不存在才写成功，这就是互斥锁。

Watch（流）=等锁队列：抢不到就 watch 该 key，删除事件一到重试；要公平锁就 watch 前一个 revision。

5. 性能工程

批处理：三个层次——多个 MsgApp 合并发送、多条 entry 打进一个 MsgApp、多个 apply 合并进一个 bbolt 事务。目标都是“一次系统调用/一次 fsync 覆盖更多请求”。

流水线：MaxInflightMsgs 让多批复制同时在途；更进一步是先发消息再写本地盘，让本地 fsync 与网络 RTT 重叠，对尾延迟影响最大。

只读不进日志：默认线性一致读走 ReadIndex——leader 先向多数派确认自己仍是 leader，拿到 read index，等本地 apply 到该 index 后直接读 MVCC，follower 也能提供。

Watch 用 gRPC 双向流：一连接挂多个 watch，按 revision 推事件，断线可从上次 revision 续传；只读 MVCC 历史，不影响写路径。

bbolt 写事务代价：单写者、页级 copy-on-write、提交要 fsync，还有 freelist 开销；所以 etcd 宁可合并 apply 提交，长期运行需 defrag 回收碎片。

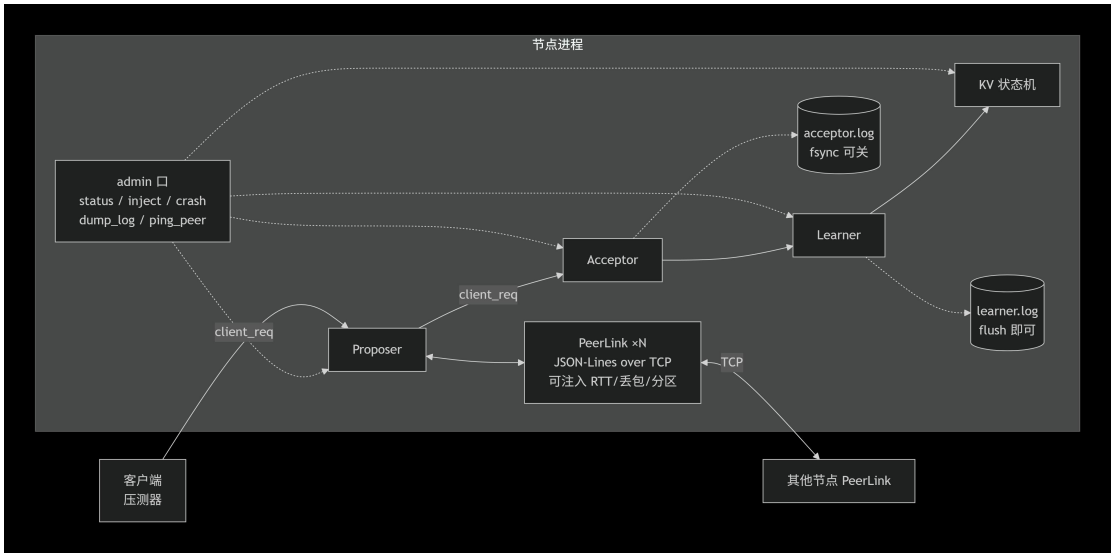
6. 架构对照

维度	Chubby	etcd
定位	粗粒度锁+小文件	强一致 KV，元数据基础设施
一致性协议	Paxos 多副本组	Raft
接口	类文件系统 RPC	gRPC KV/Watch/Lease/Txn+HTTP/JSON 网关
会话机制	Session+keepalive，过期释放锁	lease+keepalive，到期回收 key

典型用户	GFS/Bigtable/MapReduce	Kubernetes/CpreDNS/Calico
部署规模	建议 5 副本	官方 FAQ 建议低于 7 成员，生产常见 3 或者 5

四、Paxos 原型的设计与实现

1. 总体架构



2. TCP 自定义报文

报文	方向	语义
hello	连接建立	声明身份，用于区分链路与会话
prepare	Proposer → Acceptor	Phase 1a，ins=-1 表示覆盖全部实例
（应答）	Acceptor → Proposer	承诺，回报已接受值，当前承诺编号
accept	Proposer → Acceptor	Phase 2a，entries=[[slot,value],...]，支

		持批量
（应答）	Acceptor → Proposer	成功/同轮冲突/被更高编号顶掉
commit	Proposer → 全体	广播“slot 已选定”
learn_ask	Learner → 同伴	缺口回填
heartbeat	Leader → 全体	epoch+commit_index
client_req	客户端/Follower → Leader	提交命令
ping	任意	空载 RTT 测量

3. Acceptor：安全性的守门人

必须持久化 promised_id/accepted_id/accepted_value，规则有两条：

Prepare(b)：b>=promised→承诺 b（幂等），回报已接受的 (id,value)；b< promised→拒绝，并把更高的 promised 告诉提案者。

Accept(b,v)：b>=promised 且不存在同 b 不同值→接受，落盘后回复；否则→拒绝

4. 三种提案机制

机制	RTT/提案	谁能提案	实现要点
Basic Paxos	2	任意节点	每提案独立 Prepare+Accept；撞车后换新 slot+随机退避打破活锁
Multi-Paxos	1	仅 Leader	一次覆盖全部实例的 Phase 1 接管；之后只 Accept；批处理+流水线
Fast Paxos	1	任意节点	Leader 只打开轮次，任意副本把值

			直投 Acceptor；仲裁 F=⌈ 3n/4⌉

五、实验设计与结果

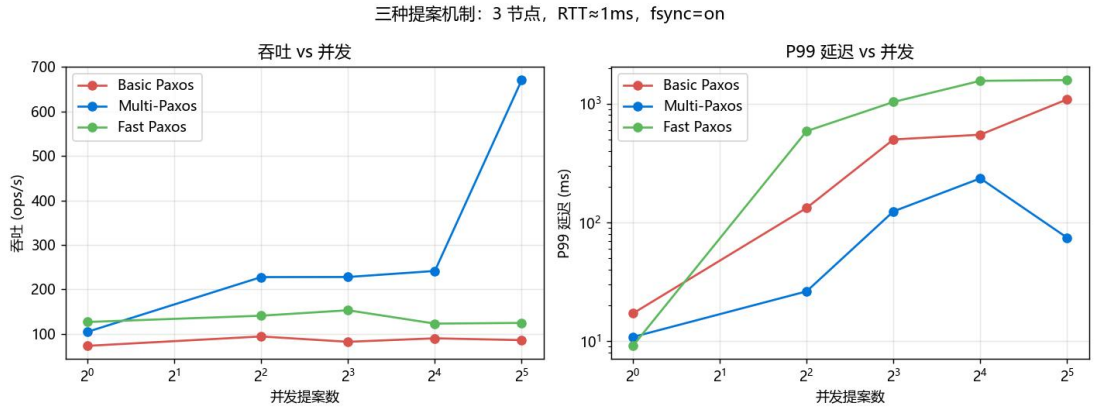
1. 实验口径

项	取值
客户端模型	闭环：N 个并发=N 条连接，每条串行请求
延迟定义	客户端发出请求，收到已被多数派确认的响应
客户端接入点	Basic/Fast：轮转全部副本；Multi-Paxos：非 Leader 副本
消息数	每个实例内的增量除以成功数
安全性	每个用例后比对所有节点的 slot 到 value 的映射
每次时长	8s，跑前 0.35s 稳定网络参数

2. 三种提案机制对比

机制	并发	吞吐(ops/s)	P50(ms)	P95(ms)	P99(ms)	错误率	消息/op	冲突率
Basic	1	72.8	12.9	14.1	17.1	0.0%	10.00	-
Basic	4	94.0	33.2	72.0	132.1	0.0%	22.11	1.1%
Basic	8	81.9	72.1	223.1	499.0	0.0%	23.66	1.5%
Basic	16	89.8	143.4	384.1	547.0	0.0%	22.82	1.3%
Basic	32	85.7	316.4	729.9	1086.6	0.0%	23.32	0.7%
Multi-Paxos	1	104.4	9.0	10.0	10.8	0.0%	8.33	-

Multi-Paxos	4	227.5	16.0	17.8	26.2	0.0%	8.15	-
Multi-Paxos	8	227.7	27.7	64.1	123.6	0.4%	9.32	-
Multi-Paxos	16	241.3	57.2	123.6	234.4	0.1%	9.97	-
Multi-Paxos	32	670.7	48.2	61.2	74.1	0.0%	4.93	-
Fast	1	126.5	7.4	8.5	9.1	0.0%	6.27	-
Fast	4	140.6	9.3	16.3	586.6	0.1%	12.00	4.3%
Fast	8	152.9	12.1	46.4	1032.7	0.3%	12.74	10.3%
Fast	16	122.9	17.7	567.0	1556.5	0.5%	15.41	22.5%
Fast	32	124.4	53.4	1067.1	1579.8	2.1%	16.31	33.9%



单并发延迟：Fast（7.4 ms）< Multi-Paxos（9.0 ms）< Basic（12.9 ms）。与理论完全一致：Basic 2 轮 RTT；Multi-Paxos 1 轮但要流经 Leader；Fast 1 轮且不流经 Leader。消息数也印证：6.27 / 8.33 / 10.00 条/op。

高并发吞吐：Multi-Paxos 一骑绝尘（670.7 ops/s）。 它是唯一消息/op 随并发下降的机制（8.33→4.93），因为它能批处理+流水线：32 条命令挤进少数几次 Accept，一次 RTT、一次 fsync 摊给多条命令。Basic/Fast 的消息/op 反而从 10.0→23.3、6.3→16.3 上升，因为撞车后要重试。

Basic Paxos 的吞吐天花板是架构性的。并发从 1 涨到 32，吞吐始终在 72.8~94.0 ops/s 之间震荡，而 P50 从 12.9 涨到 316.4 ms、P99 涨到 1086.6 ms。这就是活锁：吞吐不涨，延迟无上限地上涨。原因是每个提案都要重新走 Prepare，多个提案者用更大的编号互相顶掉。

Fast Paxos 的收益被冲突吃掉了。低并发下它最快，但冲突率从 4.3%涨到 33.9%后，吞吐停在 124~153ops/s，P99 从 9.1 ms 涨到 1579.8 ms，错误率升到 2.1%。

3. 节点数影响

机制	节点数	Q	F	吞吐 (ops/s)	P50(ms)	P99(ms)	消息/op
Multi-Paxos	3	2	3	289.5	24.8	122.4	7.06
Multi-Paxos	4	3	3	219.0	29.7	142.4	13.13
Multi-Paxos	5	3	4	238.5	30.2	86.4	15.60
Basic	3	2	3	99.1	68.3	232.1	21.80
Basic	4	3	3	76.4	79.7	416.9	42.24
Basic	5	3	4	72.4	85.9	492.2	61.95
Fast	4	3	3	153.7	15.4	1032.8	19.17
Fast	5	3	4	163.2	12.7	1027.5	25.75

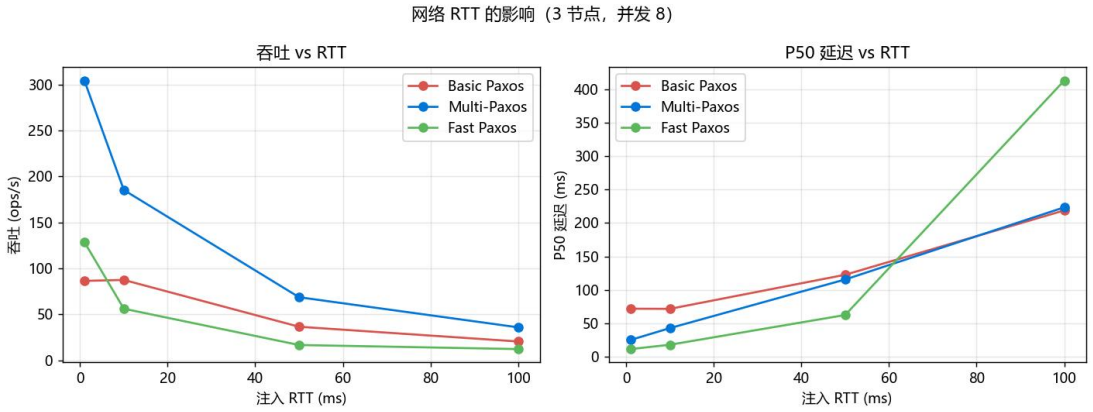
3→4 节点，多数派从 2 涨到 3，容错台数没有变化，都是 1 台，但每次提交要多等一台机器：Multi-Paxos 吞吐 289.5→219.0，消息/op7.06→13.13，P99 122.4→142.4 ms。花了更多机器、换来更差的性能和同样的容错

3→5 节点，多数派仍是 3，容错从 1 台涨到 2 台，这才是值得的扩容，Multi-Paxos 238.5 ops/s， P99 反而最低 86.4 ms，因为多了一台机器可以承担读/回包。代价是消息数继续上升。

4. RTT 扫描

机制	设定 RTT	实测 RTT	吞吐(ops/s)	P50(ms)	P99(ms)
Multi-Paxos	1	1.75	304.4	25.3	37.2
Multi-Paxos	10	10.97	185.3	43.1	54.3
Multi-Paxos	50	52.58	68.6	115.6	129.3
Multi-Paxos	100	108.36	35.8	223.4	233.9

Basic	1	1.67	86.4	72.0	453.7
Basic	10	10.78	87.4	71.8	294.6
Basic	50	50.80	36.5	122.7	749.2
Basic	100	100.97	20.4	218.8	1323.8
Fast	1	1.73	128.6	11.7	1036.9
Fast	10	10.88	56.1	18.2	1579.4
Fast	50	50.90	16.6	62.6	1736.7
Fast	100	101.01	12.2	412.6	1666.7

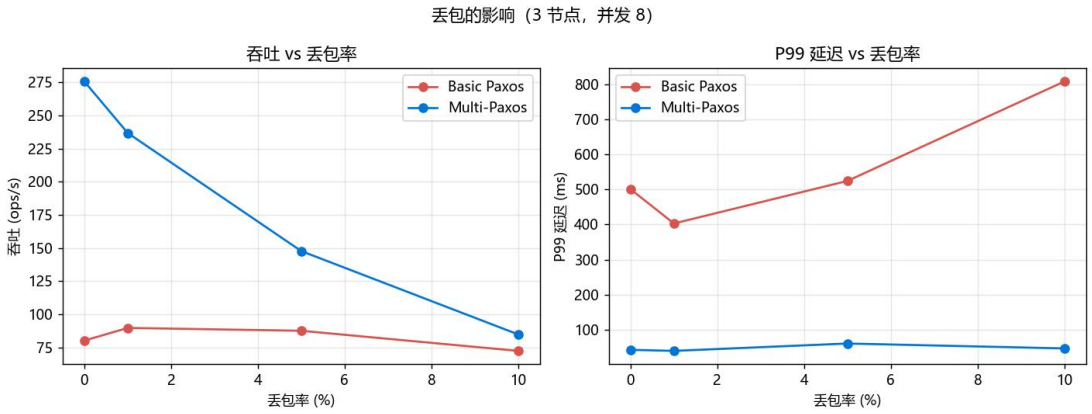


吞吐≈并发÷（k·RTT）。Multi-Paxos 的 P50 在 RTT 1→100 ms 时从 25.3 涨到 223.4，吞吐从 304.4 掉到 35.8，近似 1/RTT。Basic 在 RTT=1 ms 时被活锁压着（86.4），RTT=10 ms 时反而略升——当网络变慢，提案者撞车的概率下降，活锁缓解了。对 Basic Paxos 来说，网络快不一定全是好事。Fast 的 P99 在所有 RTT 下都是秒级（1.0~1.7 s），说明它的长尾来自冲突回退，而不是网络。

5. 丢包率扫描

机制	丢包率	吞吐 (ops/s)	P50(ms)	P99(ms)	错误率	消息/op
Multi-Paxos	0%	275.4	28.2	42.4	0.0%	6.49
Multi-Paxos	1%	236.7	27.1	39.5	1.9%	7.09
Multi-Paxos	5%	147.8	18.9	60.5	9.9%	9.55

Multi-Paxos	10%	85.0	15.4	46.5	20.0%	11.38
Basic	0%	80.3	72.8	500.8	0.0%	23.52
Basic	1%	89.8	67.8	403.0	0.0%	20.20
Basic	5%	87.7	55.8	524.4	0.0%	17.32
Basic	10%	72.5	35.1	808.0	0.0%	15.76



Multi-Paxos：丢包 0→10% 吞吐 275.4→85.0，错误率 0→20%，消息/op 6.49→11.38。临界点在 1%~5%之间。

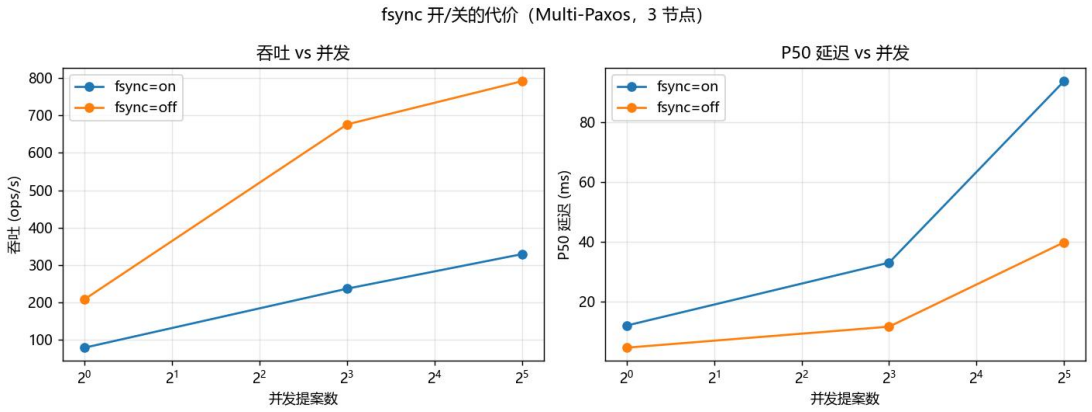
Basic：丢包对吞吐影响很小，因为 3 节点多数派只要 2 个回应，单条消息丢失常常被另外两台掩盖；但 P99 从 500.8 涨到 808.0 ms，可以看出伤害体现在长尾。

机制解释：Paxos 的重试是整轮重来，所以丢包的代价被放大；而且重试会把提案编号抬高，进一步加剧编号竞争。

6. fsync 开/关

fsync	并发	吞吐(ops/s)	P50(ms)	P99(ms)	相对提升
on	1	79.0	12.0	18.3	1
off	1	208.5	4.6	5.9	2.6
on	8	236.8	33.0	46.5	1
off	8	676.3	11.7	16.9	2.9

on	32	329.2	93.6	264.3	1
off	32	791.9	39.8	61.9	2.4



关掉 fsync 快 2.4~2.9 倍，P50 从 12.0 降到 4.6ms。本机单次 fsync 实测 2.3ms，而一次让 3 个节点都落盘的 Accept 要等最慢的那台，量级完全对得上。

--fsync off 时 Acceptor 的写只进用户态缓冲，kill -9 会丢掉未刷盘的部分，重启后 promised_all 可能比崩溃前小。一旦编号回退，同一个 slot 就可能被两个不同的值先后选定——安全性被破坏。所以这不是一个可选的性能优化，而是拿正确性换吞吐。

7. 批处理窗口

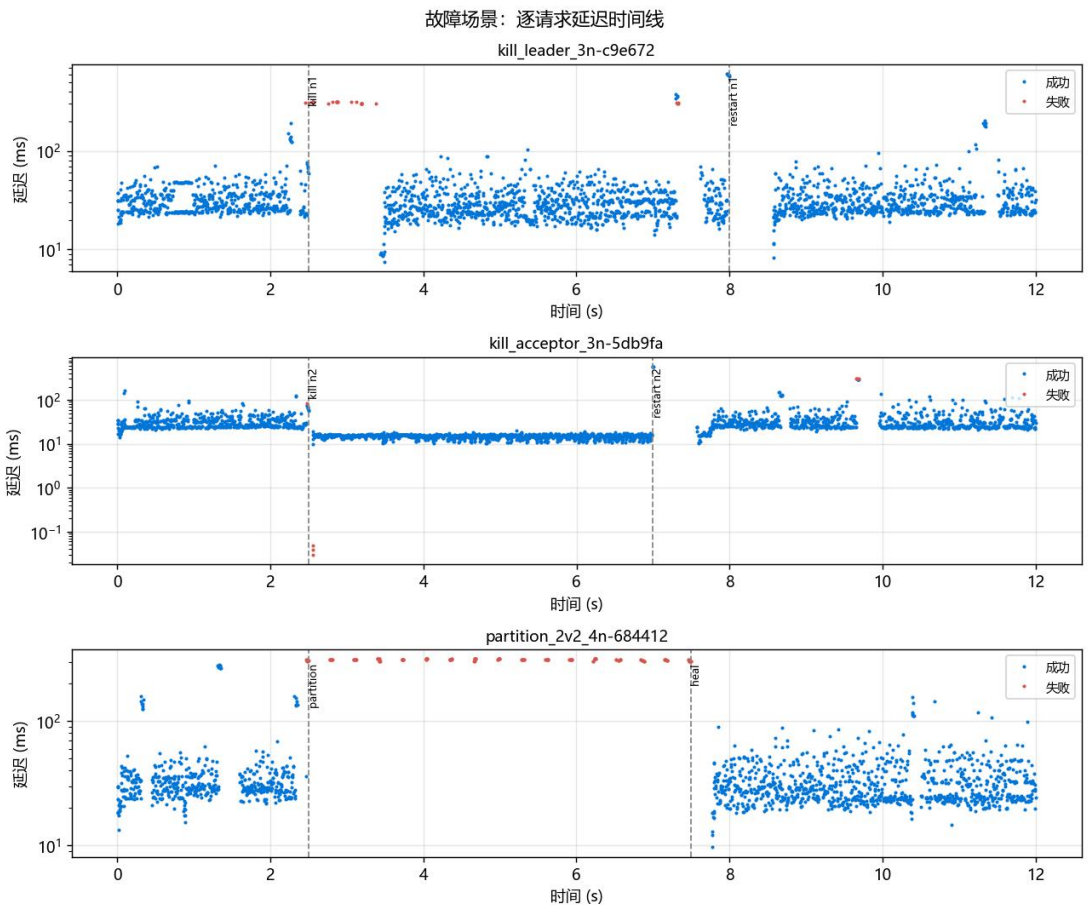
批处理窗口	吞吐(ops/s)	P50(ms)	P99(ms)	消息/op
0ms	316.7	101.7	135.3	4.75
2ms	1130.6	28.2	45.1	4.41
5ms	1026.9	29.4	54.1	4.46

仅仅等 2 ms 再发，吞吐就 3.6 倍、P50 从 101.7 降到 28.2 ms。原因是攒批让一次 RTT+一次 fsync 摊给更多命令，同时消除了每条命令都排队等一次链路往返的串行化。5 ms 窗口收益反而下降，说明批处理窗口要和到达率匹配，窗口过大只是白等。

8. 故障/分区实验

场景	机制	节	成功/总数	恢复	最长洞长	换主	安全

		点		(ms)	(ms)	次数	性冲突
kill 1 个 Acceptor（3 节点）	Multi-Paxos	3	2852/2860	63.9	335.0	3	0
kill 1 个 Acceptor（4 节点）	Multi-Paxos	4	2703/2707	89.4	2.9	4	0
kill 2 个 Acceptor（4 节点）	Multi-Paxos	4	1472/1508	70.3	5584.3	4	0
kill Leader(Multi-Paxos)	Multi-Paxos	3	2566/2595	1179.7	1177.2	5	0
kill Leader(Fast Paxos)	Fast	3	1192/1201	6796.3	5635.7	5	0
分区 2|2(4 节点)	Multi-Paxos	4	1557/1693	5294.4	5292.9	9	0
分区 2|3(客户端连接多数派)	Multi-Paxos	5	2409/2446	1350.3	1238.2	21	0
分区 2|3(客户端连接少数派)	Multi-Paxos	5	1389/4798	5264.2	5201.3	229	0



杀 1 个非 Leader 的 Acceptor，几乎无感。3 节点那 335 ms 来自恰好有一批 Accept 在途，要等 attempt_timeout 超时再换多数派，4 节点因为多数派里还有两台可直接答应，几乎没有等待。

杀 2 个 Acceptor（4 节点），完全不可用，最长黑洞变成 5584 ms，正好是从杀第二个到重启的时间，期间客户端全部失败，但已决定的值一个都没丢，重启后 70 ms 内恢复。

杀 Leader，1179.7ms 恢复。这 1.2 秒由三部分组成：选举超时 400ms（--election-ms）+随机退避 0~400 ms+一次 Phase 1（≈1RTT）+补齐未决 slot。随机退避是必须的，不加的话两个候选者会各自拿到自己+对方两票，用更大的编号互相顶掉，实测能连续换主 50+次。

Fast Paxos 杀 Leader，5636ms 黑洞，是 Multi-Paxos 的 4.8 倍。原因不是选主慢，而是 F=⌈ 3n/4⌉ =3 等于全体，一旦少了一台，快速轮在数学上永远凑不齐，只能靠 Leader 用经典轮逐个 slot 兜底。

分区 2|2，5293ms 不可用，安全性零违例。两边都凑不到多数派 3，谁都提交不了。 注意换主次数只有 9，不是选主频繁，而是根本选不出来。这正是 Paxos 的选择，宁可停摆，也不分裂。

分区 2|3，多数派一侧仍可服务，黑洞 1238ms 来自原 Leader 落在少数派，多数派要重新选主，但少数派一侧完全不可用，并且产生了 229 次换主，少数派的三台/两台在无望地反复参选。

9. 安全性汇总

指标	数值
参与比对的实验次数	60，包括 52 用例和 8 故障场景
比对过的 slot 总数	488407
初心两个 Learner 学到不同值的次数	0

所有 slot 均一致，始终保证安全性。

六、临界点分析

1. 3、4、5 节点多数派和容错能力

n	多数派 Q	容错 n-Q	Multi-Paxos 吞吐(ops/s)	P99(ms)
3	2	1	289.5	122.4
4	3	1	219.5	142.4
5	3	2	238.5	86.4

2. 为什么 4 节点不一定比 3 节点好

容错都是 1 台，但 4 节点每次提交要多等 1 台回应，实例可见 六、3，结果出现付出更多机器换更差的性能。

3. Basic Paxos 何时活锁

当并发提案者多于 2 名且同时争抢同一个 slot 时，两个提案者互相用更大的编号顶掉对方，谁也无法完成 Phase 2，结果延迟无限上涨、吞吐不涨。

4. Multi-Paxos 有 Leader 为什么快，Leader 故障恢复多久

有 Leader 可以省掉每个提案的 Phase 1，并且只有 Leader 能够做批处理；Leader 故障恢复时间由选举超时 400ms 和随机退避 0~400ms 时间构成，3 节点实测 1179.7ms。

5. Fast Paxos 冲突率多高时反而变慢

本实验里，冲突率 4.3%时 Fast 仍是低延迟之选；到时吞吐见顶，而 P99 已破 1 秒；到 33.9%时吞吐回落到 124.4，而同并发 Multi-Paxos 是 670.7，只有它的 19%。所以拐点在冲突率 10%附近，超过这个点，每次冲突都要回退，且冲突会连锁。

6. RTT 和丢包到多少时吞吐崩溃

RTT：吞吐近似∝1/RTT。Multi-Paxos 在 RTT 1→100 ms 时从 304.4 掉到 35.8。崩溃不是突变的，而是线性退化后触及客户端超时，约 2 s 才开始出现错误；

丢包：Multi-Paxos 在 1%→5%之间出现明显拐点，10% 时吞吐只剩基线的 31%，错误率 20%。

7. 故障几个节点后系统不可用

⌈ n/2⌉ 个。

8. 是否始终保证安全性

是。经过 60 次实验，488407 个 slot 的比对，0 违规。